traffic_flow 改错案例集

> 从 `/work/sunboxue/LiuLiang/traffic_flow` 的 123 个提交里，把「改错」的那批挑出来，逐个读 diff， 记下**出了什么问题、为什么错、同事怎么改的。**目的是离职后还能翻：这个项目踩过哪些坑。
>
> 数据来源：193 机，2024-09 ~ 2026-09，主要作者 fengwenxi / fengxin。 每个案例都标了 commit，原始 diff 在 `patches/*.patch` 和 `fixes/*.md` 里。

怎么用这份文档

- **按类型看**：第一部分最值得看（数据算错，不崩不报，只是结果错，最难发现）。
- **按人看**：想学 fengwenxi 的基础库功底看第三、六部分；想学 fengxin 的业务口径看第一、二、四部分。
- **面试用**：每个案例末尾有「能学到什么」和「面试可以这么说」，能直接改成"我排查过一个 xx 问题"的口播稿。但注意 —— **这些是同事的活儿，不是你的**（你只有 `6550dc1` 车头时距那一条）。
- **共 36 条**：详细展开 32 条（第一~七部分），另有 4 条在清单里标 ★ 但只做对照引用。

全景：这个项目到底出过哪些错

类型	数量	特征	危险度
数据算错	9	能编译、能跑、不报错，就是结果不对	★★★★★
时间/周期	5	周期越跑越偏、首条报文数据不全	★★★★
状态与并发	6	偶发、难复现、多线程下才现形	★★★★★
协议与输出	5	对接方读不到/读错字段	★★★
工程与构建	4	换个环境就炸	★★
可维护性	3	不是 bug，但决定了后面出多少 bug	★★★
其余速览	4	工具/配置类，改动量大但知识点集中	★★★

一句话概括这个项目的错：**大部分不是"写错语法"，而是"口径错"和"状态错"** —— 数据定义没对齐（经纬度顺序、转向位序、机动车范围）、 周期与状态的语义没对齐（窗口怎么算、标志谁来消费、0 到底表示什么）。

第一部分：数据算错

这一类最值得学。**它不崩、不报错、日志正常，只是数字全错**，而且能一路跑到现场才被发现。

1-1 经纬度传反了 —— 所有距离都是错的

commit：`19597e3`（2025-02-19，fengwenxi）

现象

所有依赖"两点距离"的指标都不对：排队长度、车头间距、区间行程时间、空间占有率。 但没有任何报错，日志里距离是个正常的正数 —— 因为**错得很"合理"**，只是数值不对。

原来的写法

```
// vec2d.hpp
double geo_distance_to(const Vec2D& other) const {
    double rad_lat1 = _y * M_PI / 180.0;        // 拿 y 当纬度
    double rad_lat2 = other.y() * M_PI / 180.0;
    double a = rad_lat1 - rad_lat2;
    double b = _x * M_PI / 180.0 - other.x() * M_PI / 180.0;   // 拿 x 当经度
    ...
}
```

为什么错

`Vec2D` 在这个项目里的构造习惯是 **`Vec2D(latitude, longitude)`**， 也就是 **`_x` 存纬度、 `_y` 存经度。**

而 `geo_distance_to` 里的写法（`_y` 当纬度、 `_x` 当经度）是**教科书公式的默认假设**： 一般坐标系里 x 是经度、y 是纬度。写这段的人按通用习惯写，但项目的 `Vec2D` 是反的。

改完：

```
double rad_lat1 = _x * M_PI / 180.0;        // x 才是纬度
double rad_lat2 = other.x() * M_PI / 180.0;
double a = rad_lat1 - rad_lat2;
double b = _y * M_PI / 180.0 - other.y() * M_PI / 180.0;
```

怎么发现的

看同一提交里的测试文件：

```
// 改前
Vec2D vec1(116.636644227, 40.045585559);   // 经度在前
// 改后
Vec2D vec1(30.52017825827854, 119.96812001698534);   // 纬度 30.52 在前
```

北京某路口的坐标 `(116.63, 40.04)` 被换成了杭州附近的 `(30.52, 119.96)` —— **测试数据跟着一起改了，说明是拿真实设备的新坐标重算了一遍才对上。**

能学到什么

- 这类"参数顺序"错误**编译器和静态检查都抓不到**，因为两个参数都是 `double`。
- 防线只有两条：① 类型要区分（`Latitude` / `Longitude` 各做一个强类型，或者干脆 `GeoPoint` 结构体）； ② 单元测试用**真实坐标**断言距离，而不是"跑通就行"。
- 代码里其实留了注释 `// TODO: Encapsulate into a derived class GeoPoint` —— 写的人早知道该封装，只是没做。

> **面试可以这么说**：所有坐标系相关的 bug 我都坚持一条 —— 单位和顺序必须写进类型里， 不能靠变量名。经纬度都用 double 是最危险的，因为传反了编译器毫无办法。

1-2 车头时距用「中心点」算车身 —— 正常车队全被判成断队

commit：`997de52`（2026-09-08，fengxin）

现象

排队指标里的"排队车辆数"永远只数出 **1 辆**，实际路口明明排了一长串。 调整门限参数也没用，改大改小都一样。

原来的写法

```
double prev_dist = 0.0;
for (const auto& track : queued_objects) {
    const double dist = track.track.back().position.geo_distance_to(stop_mid);
    const double body = track.object_length > 0.0f ? track.object_length : 4.5f;
    if (dist - prev_dist > _rules.entry_gap_body_ratio * body) {
        break;                          // 间距超了 1 个车身 → 认为队伍断了
    }
    queue_chain.push_back(track);
    prev_dist = dist;
}
```

为什么错

`position` 是目标的**中心点**。两辆停稳的车，中心点间距 ≈ **车长 + 车间隙（约 1.5m）**， 而门限是 `1.0 × 自身车长`。所以：

```
中心间距（4.75 + 1.5 = 6.25m） > 门限（1.0 × 4.75 = 4.75m） → 判为断队
```

正常停好的车队，第一辆之后必然断开，于是永远只数出 1 辆。

改法

把"中心点间距"换成"车头-车尾间距"：

```
double prev_rear_m = 0.0;
bool is_head = true;
for (const auto& track : queued_objects) {
    const double body = body_length(track);                 // object_length>0 就用它，否则 4.5m
    const double center_m = track.track.back().position.geo_distance_to(stop_mid);
    // 头车量到停止线；后面每辆量到前一辆的车尾；已越过停止线的车头按 0 计
    const double gap_m = std::max(center_m - body / 2.0 - (is_head ? 0.0 : prev_rear_m), 0.0);
    if (gap_m > _rules.entry_gap_body_ratio * body) {
        break;
    }
    queue_chain.push_back(track);
    prev_rear_m = center_m + body / 2.0;                    // 记住本车车尾
    is_head = false;
}
```

三处关键：

1. `center - body/2` = 本车车头；`prev_rear_m` = 前车车尾。两者之差才是真正的间隙。
2. `std::max(..., 0.0)` —— 车头已经越过停止线时，`center - body/2` 可能为负， 夹到 0 表示"贴住了"，不能算出负间距（负间距会让 `gap > 门限` 恒不成立，反而全算进队）。
3. 兜底车长从常量改成 `body_length(track)`： 上游 RSM 每条目标都带 `length`（注释写"回放实测 9071/9071 有值"）， 只有长度为 0 的异常目标才用 4.5m。

顺带还修了 `QueueLength`：

```
// 改前：队尾车的中心点到停止线
_latest_result["QueueLength"] = queue_chain.back().track.back().position.geo_distance_to(stop_mid);
// 改后：队尾车的车尾到停止线
_latest_result["QueueLength"] = static_cast<float>(chain_tail_rear_m);
```

需求表写的是"排队车辆**最远处**距离停止线的距离"，最远处是**队尾车尾**，不是中心。

能学到什么

- **指标定义要和几何量的物理意义对齐。**"车距"到底是中心距、车头距还是车尾距，差一个车身。
- 出问题时先怀疑**门限公式的坐标系**，再怀疑门限值本身。
- 一条注释写得很到位：

> `lon/lat 是目标中心，直接用中心间距当「车间距」会把车身本身也算进间距里， 正常停稳的车队（中心间隔 = 车长 + 约 1.5m）必然超限、只数得出一辆车，故先换成车头/车尾位置。` —— 这种"把推理过程写进注释"的习惯，值得抄。

面试可以这么说：我排查过一个"排队永远只有 1 辆车"的问题，根因是拿目标中心点算车间距， 等于把车身长度也算进了间隙，正常车队必然判定断裂。改成车头对前车车尾之后才对上。

1-3 左转右转的位序搞反 —— 流向数据整体错位

commit：`c2327c7`（2026-09-09，fengxin）

现象

按流向（左转/直行/右转）拆分的流量、排队数据**互相串了**： 左转车道的数据跑到右转里去，反之亦然。总量对，但分项全错。

原来的写法

```
bool has_left  = (turn_dir_no.size() >= 2 && turn_dir_no[1] == '1');
bool has_right = (turn_dir_no.size() >= 4 && turn_dir_no[3] == '1');
```

为什么错

`turn_dir_no` 是 4 位字符串，注释里明确了位序：

> 位序从左到右是 **掉头 | 右转 | 直行 | 左转** （`0001`=左转、`0010`=直行、`0100`=右转、`1000`=掉头）

所以 **下标 1 是右转、下标 3 是左转**。原代码刚好取反：

字段	原代码取的位	实际含义	结果
`has_left`	`[1]`	右转	左转判定成了右转
`has_right`	`[3]`	左转	右转判定成了左转

改法

```
bool has_left  = (turn_dir_no.size() >= 4 && turn_dir_no[3] == '1');   // 下标 3 = 左转
bool has_right = (turn_dir_no.size() >= 2 && turn_dir_no[1] == '1');   // 下标 1 = 右转
```

同一次提交还把配置文件里的 `turn_dir_no` 从 `"0000"` 改成 `"0010"`（直行）：

```
-  "turn_dir_no": "0000"
+  "turn_dir_no": "0010"
```

`0000` 表示**没有任何转向**，那条车道根本不会被分摊到任何流向 —— 数据直接消失。 所以这次是"代码位序错" + "配置转向码没填"两个问题叠在一起。

能学到什么

- 位域/位序这种"隐式约定"，必须有一个**唯一的常量或注释**兜住，不能靠记。 这个项目是用注释写在 `spatial_aggregator.cpp` 里，其实有点散。
- 排查思路值得学：**总量对、分项错 → 一定是拆分/分摊环节**，不要去看累加逻辑。
- 同一个人的提交里还有一条相关注释：

  > `掉头无流向枚举（需求表 R2 已删 TurnAround），故不参与分摊。` —— 位里留了掉头的位，但枚举里没有，所以分摊时要显式跳过。

> **面试可以这么说**：流向数据错位这类问题，判断方向很快 —— 总量对分项错就是分摊映射错了， 不用查累加。我们那次是转向码的位序索引反了，1 和 3 互换了。

1-4 Volume 把自行车算进了机动车

commit：`84b7d07`（2025-02-18，fengwenxi）

现象

路口机动车流量偏高，而且非机动车、行人多的路口偏高得更明显。

原来的写法

```
// volume_detector.cpp
_volume = _volume1 + _volume2 + _volume3 + _volume4 + _volume5;   // 全部加起来
```

```
// traffic_flow_detector.h —— 分类集合定义
const static std::set<InnerType> g_motor_vehicle_types = {
    InnerType::CAR, InnerType::BUS, InnerType::TRUCK,
    InnerType::BICYCLE, InnerType::TRICYCLE      // ← 自行车、三轮车被归进"机动车"
};
const static std::set<InnerType> g_non_motor_vehicle_types = { InnerType::PEDESTRIAN };
```

为什么错

两层错，叠在一起：

1. 协议里 `Volume` 是**机动车总流量**，`Volume1` 是行人、`Volume2` 是非机动车， 只有 `Volume3~5` 才是机动车（小车/面包车/大车）。全加等于把行人和非机动车也算成车。
2. 分类集合本身也错了：`BICYCLE` / `TRICYCLE` 被放进 `g_motor_vehicle_types`， `g_non_motor_vehicle_types` 只剩下行人。**所以凡是用这个集合过滤的地方，自行车都被当机动车处理。**

改法

```
_volume = _volume3 + _volume4 + _volume5;        // 只算机动车三类
```

```
const static std::set<InnerType> g_motor_vehicle_types = { InnerType::CAR, InnerType::BUS, InnerType::TRUCK };
const static std::set<InnerType> g_non_motor_vehicle_types = {
    InnerType::PEDESTRIAN, InnerType::BICYCLE, InnerType::TRICYCLE
};
```

后续演进（重要）

半年后（`90982e3`，2026-09）这套逻辑又被重构了一次，从"集合过滤"改成**枚举分型**：

```
enum class VolumeCategory : int {
    NONE = 0, PEDESTRIAN = 1, NON_MOTOR = 2, CAR_SMALL = 3, VAN = 4, LARGE = 5
};
constexpr std::array<const char*, 5> kVolumeDetailKeys = {"Volume1","Volume2","Volume3","Volume4","Volume5"};

// 计数容器从单个 int 变成 5 格数组，下标 0~4 对应 Volume1~5
struct LaneAcc { std::array<int, 5> counts{}; float pcu = 0.0f; nlohmann::json meta; };

// 输出时：机动车总流量 = 尾部连续三类之和
int motor_volume = 0;
for (size_t i = static_cast<int>(VolumeCategory::CAR_SMALL) - 1; i < acc.counts.size(); ++i) {
    motor_volume += acc.counts[i];
}
entry["Volume"] = motor_volume;
```

为什么这么改：用 `enum` 的数值直接当协议字段编号（`PEDESTRIAN=1` → `Volume1`），分类和协议字段**一一对应、不可能错位**；而之前"集合 + 五个独立计数器"是两套东西，容易不一致。

能学到什么

- **"总量对不上"先查分子范围**，别查加法。这里的错是"谁算机动车"定义错了。
- 用**枚举值 = 协议字段编号**是一种很强的防错手法：数字本身有业务含义，不用再维护一张映射表。
- 反过来说，`std::set<InnerType>` 这种"运行时集合"做分类，编译器不会帮你检查遗漏；换成 `switch` 覆盖所有枚举 + `default` 报警，才能保证新类型不会漏。

> **面试可以这么说**：我们流量统计出过一个"机动车偏高"的问题，根因是分类集合里把自行车三轮车错放进了机动车，而且总流量把行人也加了进去。后来重构成枚举分型，枚举值直接等于协议字段编号，从结构上避免错位。

1-5 占用率：拿「队尾到停止线」当「占用长度」

commit：`19597e3`（2025-02-19，fengwenxi）同一提交里的另一处

原来的写法

```
// 空间占有率 = 车道被车辆占据的长度 / 车道总长
auto pos1 = lane_objects[lane_objects.size() - 1].track.back().position;
float occupied_length = pos1.geo_distance_to(stop_line_middle_point);
space_occupancy = occupied_length / _lane_length;
```

为什么错

这算的是"**最远那辆车离停止线多远"，即排队长度**，不是"车辆占据的总长度"。一条 100m 的车道上排了 3 辆 4.75m 的车，占用率应该是 `14.25/100 ≈ 14%`，但按原算法会算出 `90/100 = 90%` —— **完全不同的物理量**。

改法

```
float occupied_length_sum = 0.0f;
for (const auto& one_object : lane_objects) {
    occupied_length_sum += one_object.object_length;      // 累加每辆车自身长度
}
space_occupancy = occupied_length_sum / _lane_length;
```

有意思的是：**这段"正确写法"其实原来是被注释掉的**，代码里同时存在两种实现，只是选了错的那个。 改的时候正好反过来，把对的启用、错的注释掉。

能学到什么

- **指标名字和公式要对得上。**"空间占有率"是密度指标（长度比），"排队长度"是距离指标， 长得很像但物理含义完全不同。
- 代码里注释掉一版正确实现、只留错误实现 —— 这种"两套逻辑并存"是隐患， 审查时看到注释掉的核心算法一定要问"为什么"。

1-6 用 `inner_id` 当轨迹键 —— 跨帧串车

commit：`7f71330`（2025-02-18，fengwenxi）

原来的写法

```
auto& object_track_info = _object_tracks[one_object.inner_id];    // 拿 inner_id 当 map 键
if (object_track_info.id != one_object.inner_id) {
    object_track_info.id = one_object.inner_id;
}
```

```
// 注释里写的是
// key: inner_id, value: track info
```

为什么错

`inner_id` 是**感知内部**的编号（`uint64_t`），只在单帧/单传感器内唯一， 跨帧、跨传感器可能被复用或重新分配。拿它当"这辆车是谁"的键会出两种问题：

1. **同一辆车换帧后 inner_id 变了** → 被判成新车，轨迹断裂，重复计数。
2. **不同车在不同帧拿到同一个 inner_id** → 两辆车的轨迹混成一条，速度和距离全错。

改法

```
auto& object_track_info = _object_tracks[one_object.id];          // 用全局稳定的 id
if (object_track_info.id != one_object.id) {
    object_track_info.id = one_object.id;
}
// key: object_id, value: track info
```

`id` 是 `std::string`（融合后的全局目标 ID），跨帧稳定。

能学到什么

- **凡是"用它当 map key 追踪一个实体"的字段，必须确认它的生命周期。** 内部临时编号、帧内索引、数组下标都不能当实体 ID。
- 这个项目里同时存在 `id` 和 `inner_id` 两个字段，名字太像， 新人不看注释就会选错 —— 这也是为什么后来很多地方干脆只用 `id`。

1-7 哨兵值 `-1` 与 `0` 的口径之争

commit：`1f4b516`（2026-09-16，fengxin）

背景

"没有样本"怎么表示？这个项目争论过，而且**结论是反直觉的。**

原来的写法（技术上更严谨）

```
// 窗口内没有完整通过本区间的有效车辆时写 -1，不能写 0：
// 0 表示「一辆车都没停过」，和「没车可测」是两回事。
entry["AvgStopTimes"] = _settled_vehicles > 0 ? ... : -1.0;
```

同样的写法出现在多个检测器里：

字段	无样本时的原值	理由
`AvgStopTimes`	-1	0 = 一辆车都没停过
`DelayTime`	-1	0 = 一辆车都没延误
`Headway` / `Gap`	-1	0 = 两车同一毫秒过线（真实极端值）
`HeadwayDistance`	-1	0 = 车头贴车尾（有物理含义）

改成了什么

全部改成 `0`：

```
// 无样本写 0（全模块哨兵统一为 0），与「一辆车都没延误」不可区分。
entry["DelayTime"] = _settled_vehicles > 0 ? _delay_sum_s / _settled_vehicles : 0.0;
```

为什么"退步"了

因为**全模块 + 上下游协议要统一。**混用 `-1` 和 `0` 会导致：

- 下游做统计时要对每个字段单独判断"这个 -1 是不是哨兵"；
- 有些字段（计数类）本来就没有负数，-1 突兀；
- 协议里其它字段已经用 0 了，不一致比"精确"危害更大。

代价是显式承认的：注释写"与真实极端值**不可区分**"， 下游要判断有没有样本，**得去看明细串的元素个数**（比如 `HeadTimeDiffDetail` 是不是只有一个 `0`）。

同时这次还把 `BranchType` 从 `-1`（表示斑马线这类非进出口断面）改成 `0`， `BranchID` 从字符串 tag 改成 **8 方向码**（1北 2东 3南 4西 5东北 6东南 7西南 8西北）：

```
entry["BranchID"]   = static_cast<int>(found ? find_it->second.direction : DirectionCode::DEFAULT);
entry["BranchType"] = (found && find_it->second.type == LaneType::EXIT_SECTION) ? 1 : 0;
// 反查不到时方向填 0（DEFAULT），与 NodeID/UpstreamCrossID 填空串一致：宁可显式无值，不猜方向。
```

能学到什么

- **"更精确"和"更一致"冲突时，工程上通常选一致。**但前提是把代价写进注释、并且给下游留替代判据。
- 这次改动的注释是我见过最坦白的：

  > 无样本写 `0`（全模块哨兵统一为 `0`）：代价是与「两车同一毫秒过线」的真实极端值不可区分， 下游要判断有没有样本得看 `HeadTimeDiffDetail` 的元素个数。

- 反过来，如果让你设计，正确做法可能是**加一个 `HasSample` 布尔字段**， 而不是在数值上编码"有没有"—— 数值字段不该承载两种语义。

> **面试可以这么说**：我们统一过哨兵值。原来无样本写 -1 更严谨，但和协议其它字段不一致， 下游每个字段都要单独判哨兵。最后统一成 0，代价是和真实极端值不可区分， 所以给下游留了明细串长度作为判据。这件事让我觉得，"有没有值"应该单独一个字段，不该塞进数值里。

1-8 车尾时刻的换算 —— 除号写错了位置，Gap 小了 12.96 倍

commit：`3528089`（2025-02-20，fengwenxi）引入；`eb14716`（2026-09-03）新架构重写时改对

现象

车头时距检测器输出两个量：

- `Headway`：前车**车头**到后车**车头**的时间差
- `Gap`：前车**车尾**到后车**车头**的时间差（净间隔，排队消散能力看这个）

按物理意义， `Gap` 应该比 `Headway` 小"一个车身通过停止线的时间"，30 km/h 下大约小 0.5 秒。 现场实测只小了 **0.04 秒**左右。

原来的写法

```
// 车尾离开停止线的时刻 = 车头到达时刻 + 车身通过时间
_last_tail_arrival_timestamp =
    one_track[one_track.size() - 1].timestamp
    + one_object_track.second.object_length
      / one_track[one_track.size() - 1].speed
      / MS_TO_KMH
      * SECOND_TO_MILLISECOND;
```

为什么错

`MS_TO_KMH = 3.6`（定义在 `src/common/config.h`），而 `speed` 字段单位是 **km/h**（`src/common/perception_data.h:205` 注释写着 `speed // in km/h`）。

按量纲推一遍：

```
车长[m] / 速度[m/s]        = 秒
速度[m/s] = 速度[km/h] / 3.6
所以      = 车长 / (speed / 3.6) * 1000   [ms]
```

代码写成 `车长 / speed / 3.6 * 1000`，等于把 `÷3.6` 放到了括号**外面** —— 一除一乘搞反，结果偏小 `3.6² = 12.96` 倍。

举例（车长 4.5 m、30 km/h）：

	车身通过时间
真实值	4.5 / 8.33 = **540 ms**
代码算出	4.5 / 30 / 3.6 × 1000 = **41.7 ms**

改法

新架构 `AccumulateHeadwayDetector` 重写时写对了，差别只在括号：

```
uint64_t tail_time = current_time + (uint64_t)(obj.length / (obj.speed / 3.6) * 1000);
//                                             ^^^^^^^^^^^^^^^^^^ 先把 km/h 化成 m/s
```

怎么发现的

这种错不崩、不报错、日志也正常。只能靠"物理意义推出来的期望值"和实际输出对一遍： `Gap` 该比 `Headway` 小一个车身时间，实测只小几十毫秒 —— 差一个数量级，才回头看公式。

能学到什么

- **单位换算别在脑子里算，先写量纲式再抄成代码**：`[m] / ([km/h] / 3.6) * 1000`。 只要式子里有两个以上运算，就用括号把"换算"圈起来。
- **宏名只说明"是什么"，不说明"怎么用"**。`MS_TO_KMH` 看着像个系数，实际该除还是该乘， 宏本身不会提醒你。这里用裸 `3.6` 反而更难错 —— 因为它逼你想"我在除什么"。
- **凡是"两个量之间应该差一个已知量"的地方，就写一条自检**： `Gap ≈ Headway − 车身通过时间`。这类恒等式是查口径错最便宜的手段，比看日志有效。
- 与 1-2（中心点算车间距）同型：**物理模型是对的，表达式写歪了。**

> **面试可以这么说**：我们算 Gap，就是前车尾到后车头的净间隔，要用车长除以速度算车身通过时间。 这里踩过一个坑：速度是 km/h，换成 m/s 要除 3.6，结果括号位置放错，变成又除了一次， 算出来小了 12.96 倍。它不崩不报错，只是 Gap 一直偏小，最后是拿物理意义去对才发现的。 所以我现在写单位换算，会先写量纲式再抄成代码。

1-9 平均车距/占用率在结算那一帧现算 —— 等于只统计了最后 100 毫秒

commit：`3528089`（2025-02-20，fengwenxi）

原来的写法

```
void HeadwayDetector::calculate_once(nlohmann::json& result_j) {
    // 结算的那一刻，才去遍历"当前帧"，算车道内相邻车辆间距的平均
    std::vector<TrackInfo> lane_objects;
    for (auto& one_object : _current_frame_result.traffic_objects) { /* 收集车道内的车 */ }
    std::sort(lane_objects.begin(), lane_objects.end(), /* 按离停止线远近排序 */);

    float distance = 0.0f;
    if (lane_objects.size() > 1) {
        for (size_t i = 1; i < lane_objects.size(); ++i) {
            distance += 相邻两车距离;
        }
        distance /= (lane_objects.size() - 1);
    } else {
        distance = -1.0f;   // 车里 0~1 辆 → 直接给"无值"
    }
    result_j["Distance"] = distance;
}
```

为什么错

三个问题叠在一起：

1. **这是空间快照，不是周期统计。** 周期可能是 5 分钟，但只用了最后 1 帧（约 100 ms） 的车道状态去代表 5 分钟。
2. **结果被"结算瞬间恰好排了几辆车"决定。** 周期末刚好绿灯放空 → 车道里 0~1 辆 → 输出 `-1`，整整 5 分钟的统计白做。反过来周期末刚好堵着 → 输出的是拥堵车距。
3. **它算的其实是"排队间距"，不是"车头间距"。** 只有堵车时车道里才会同时有多辆车， 所以这个平均值本质上是"拥堵状态下的车距"，不能代表整个周期。

改法

改成**每帧累积、结算时只做除法**：

```
// update() 每帧末尾：车道内有 n 辆车就累加 n-1 个相邻间距
for (size_t i = 1; i < lane_objects.size(); ++i) {
    ++_headway_distance_count;
    _headway_distance_sum += 相邻两车距离;
}

// calculate_once 只除
float distance = -1.0f;
if (_headway_distance_count >= 1) {
    distance = _headway_distance_sum / _headway_distance_count;
}

// 结算完清零，进入下一个周期
_headway_distance_count = 0;
_headway_distance_sum = 0.0f;
```

同一次提交里，占用率也做了同样的改造，而且多了个细节：

```
// 每帧：把本帧车道内所有车的车长累加；帧数无条件 +1（空帧也 +1）
for (const auto& one_object : lane_objects) {
    _occupied_length_sum += one_object.object_length;
}
++_space_occupied_count;

// 结算：
space_occupancy = _occupied_length_sum / _lane_length / _space_occupied_count;
```

关键在 `++_space_occupied_count` **空帧也要加**：

- 分母是**总帧数** → 得到"周期内平均占用率"，可以是 0，符合直觉；
- 如果只统计"有车的帧" → 得到的是"有车时刻的平均占用率"，**永远不为 0**，等于把"道路空闲" 这个最重要的状态抹掉了。

能学到什么

- **看到"周期统计量"，先问一句：分子分母是按帧累加，还是结算时现算？** 现算 = 只看最后一帧。
- **分母口径必须和分子一致。** 分子空帧贡献 0，那空帧也要占一个分母位，否则平均值被抬高。 这是"平均"最常见的错法 —— 不写 `count` 的循环，最后拿 `size()` 以外的数当分母。
- **量的物理含义要跟"什么时候采样"匹配。** "平均车距"是时间平均还是空间平均，决定你该在 哪一层累加。这里改完其实是"周期内所有相邻车对的平均"，仍然不是严格的时间平均， 但比"最后 1 帧的快照"好得多 —— 工程上够用。
- 与 2-1（相对窗口漂移）、3-3（bool 被多消费者抢）同属一类：**状态的生命周期没想清楚。**

> **面试可以这么说**：我们平均车距原来是结算时遍历当前帧算的，等于用最后 100 毫秒代表 5 分钟， 而且周期末刚好放空就直接输出无值，整周期白算。后来改成每帧累积、结算时除次数。 占用率是同一个坑，要点是分母必须按总帧数算，空帧也要占一个分母位， 不然算出来是"有车时的占用率"，永远不为 0，把道路空闲这个状态抹掉了。

第二部分：时间与周期

这一类的共同点：**跑一天看不出来，跑一周就明显了。**

2-1 相对窗口漂移 —— 5 分钟变成 5 分钟+Δ，而且只增不减

commit：`90982e3`（2026-09-07，fengxin）

原来的写法

```
// OutputChannel::should_output
return (current_time - last_output_time) >= interval_ms;
```

为什么错

窗口起点是"上次**实际上报**的时刻"，而实际上报时刻 = 帧到达时刻。 帧到达本身有抖动（±几十毫秒），每次抖动都会**累积**进下一个周期：

```
第1次：0        → 300.03s 上报（晚 30ms）
第2次：300.03s  → 600.08s 上报（又晚 50ms）
第3次：600.08s  → 900.15s 上报（又晚 70ms）
...
```

于是 5 分钟的周期**越来越长，且只增不减**，一天下来能漂出几分钟。 更麻烦的是：`StartTime` / `StopTime` 里带的是"帧到达的零头"，不是整齐的时刻。

改法：改成绝对栅格

```
// 触发判定走 interval_ms 的绝对栅格：只有帧时间戳跨入新的栅格槽才上报，
// 帧到达的抖动不会累积进周期（5min 不会变成 5min+Δ 且只增不减）。
const uint64_t grid_open_ms = current_time / interval_ms * interval_ms;
if (grid_open_ms <= window_open_ms) {
    return false;
}
// 标签取栅格时刻：帧落在槽内的哪一拍是到达抖动，不该写进 StartTime/StopTime
window_start_ms = window_open_ms;
window_stop_ms  = grid_open_ms;
window_open_ms  = grid_open_ms;
```

`current_time / interval_ms * interval_ms` 就是"把当前时刻向下取整到最近的栅格边界"（整数除法截断），比如 `interval=300000ms` 时，`16:43:27` 落到 `16:40:00` 这个槽。

抖动不再累积：每个槽的起点由绝对时间决定，跟上一帧什么时候到完全无关。

附带解决：断流跨槽

```
const uint64_t merged_windows = current_time / interval_ms - window_open_ms / interval_ms;
if (merged_windows > 1) {
    std::cerr << "[OutputChannel] " << cycle_type << " merged " << merged_windows
              << " windows of " << interval_ms << "ms, no frame in between" << std::endl;
}
```

断流几分钟后恢复，中间跨过的槽**不补发**（检测器状态本来就跨流保留）， 并入本次窗口一次性上报，但**打告警**——否则运维会以为丢报文了。

能学到什么

- **周期性任务用"绝对时间取整"，不要用"距上次的时间差"**。这是定时器设计的通用原则， 从 cron 到监控采样都成立。
- 反过来，如果业务要求"严格等间隔"，就必须接受"首帧对齐"的代价； 如果要求"对齐墙上时间"，就必然会有"第一个窗口不完整"（见下一条）。
- 一个判据：**只要周期值要出现在报文里（`Cycle=300`），就必须用栅格**， 否则你标 300 秒、实际 300.4 秒，长期对不上。

> **面试可以这么说**：我们 5 分钟周期的报文时间戳会越跑越偏。原因是按"距上次上报的差值"触发， 帧到达的抖动每次累加。改成按绝对时间取整到 5 分钟栅格，抖动就不累积了。 代价是启动时第一段窗口不完整，需要额外丢掉。

2-2 启动头段的残窗 —— 第一条报文数据只有一半却标着 Cycle=300

commit：`0e6486b`（2026-09-08，fengxin）

现象

模块启动后第一条 5 分钟报文，里面只有 2 分钟的数据，但 `Cycle` 字段照样写 300。 下游按 300 秒去折算流量，结果偏低 40%。

为什么

接上一条：改成栅格之后，如果**启动时刻落在槽中间**（比如 `16:43:27` 启动）， 那么第一个"上报时刻"是 `16:45:00`，但实际统计窗口只有 1 分 33 秒。

改法：显式标记残窗并丢弃

```
bool head_partial = false;

if (!initialized) {
    initialized = true;
    window_stop_ms = current_time;
    if (is_signal) {
        window_open_ms = current_time;
        head_partial = false;
    } else if (align_to_grid) {
        const uint64_t grid = current_time / interval_ms * interval_ms;
        window_open_ms = grid;
        // 首帧正好踩在栅格边界上时，首个窗口本来就是完整的，不算残窗
        head_partial = (current_time != grid);
    }
    ...
}

// ...
if (head_partial) {
    head_partial = false;
    drain_accumulators();      // 把累积状态丢掉，不出报文
    return false;
}
```

两个细节值得学：

1. `head_partial = (current_time != grid)` —— 如果启动瞬间正好在边界上，首窗就是完整的， 不能一刀切地丢。
2. `drain_accumulators()` —— 不是简单 return，而是调一次 `calculate_once()` 把累积器清零。 否则这 1 分 33 秒的数据会并进下一个完整窗口，等于**污染下一条报文**。

```
// 丢弃当前累积状态：窗口标签之外的数据不该被算进下一条报文。
void drain_accumulators() {
    for (auto& detector : detectors) {
        detector->calculate_once();
    }
}
```

利用"`calculate_once` 算完即清零"的既有语义来丢数据——**借力现有语义，不新增清零接口。**

能学到什么

- 周期系统一定有"第一个窗口"的问题。要么丢（本文做法），要么标出来（加 `Partial=true` 字段）。 最差的是假装完整。
- "丢数据"要丢干净：只跳过上报、不清累积器，等于把脏数据推到下个窗口。

2-3 信号周期靠定时器模拟 —— 把真实链路的问题盖掉了

commit：`0e6486b`（2026-09-08，fengxin）核心改动

原来有什么

`SignalPhaseManager` 里有一整个后台线程：

```
void SignalPhaseManager::start_simulation(uint64_t interval_ms) {
    _sim_running = true;
    _sim_thread = std::make_unique<std::thread>([this, interval_ms]() {
        while (_sim_running) {
            std::this_thread::sleep_for(std::chrono::milliseconds(interval_ms));
            if (!_sim_running) break;
            std::lock_guard<std::mutex> lock(_mutex);
            _cycle_boundary = true;                        // 每 N 秒假装"一个新的信号周期"
            std::cout << "[SignalSim] Simulated cycle boundary TRIGGERED" << std::endl;
        }
    });
}
```

配置里有个 `simulate_interval_ms`，配上就每 N 秒伪造一次周期边界。

为什么这是错

1. **模拟出来的边界和真实红绿灯毫无关系。**现场调试时看着"信号周期级"有数据输出， 其实是定时器在出，会让人误判整条信号链路是通的。
2. 真实链路（DDS 信号灯 topic → 相位状态 → 非绿转绿）坏了也发现不了。
3. 这个开关一旦被带进生产配置，报文的 `Cycle` 语义就彻底说不清了。

改法：整段删掉，只认真实信号

```
// 边界由真实信号报文推进：参考相位非绿转绿。没有信号数据时本通道就是不出数，不做定时模拟。
const uint64_t boundary_count = spm ? spm->cycle_boundary_count() : 0;
if (boundary_count == seen_cycle_boundary) {
    return false;
}
seen_cycle_boundary = boundary_count;
```

`simulate_interval_ms` 字段、`start_simulation()`、`_sim_thread`、`_sim_running` 全部删除。

同时补了配置校验：

```
if (cycle_config.cycle_type == "signal_cycle") {
    if (cycle_config.reference_phase_id >= 0) {
        _signal_phase_manager->set_reference_phase_id(cycle_config.reference_phase_id);
    } else {
        std::cerr << "[BuildDetectors] Channel [signal_cycle] has no reference_phase_id, "
                     "it will never output without a signal phase to watch" << std::endl;
    }
}
```

—— **配了信号周期通道但没配参考相位，就明确告警"永远不会出数"**，而不是静默无输出。

能学到什么

- **"模拟/兜底/假数据"开关是排查能力的敌人。**短期能跑通演示，长期掩盖真问题。 正确做法是：没数据就报"没数据"，而不是造一个"像有数据"的东西。

- 这条和项目里的注释是一致的：

> 没有信号数据时本通道就是不出数，不做定时模拟。

- 顺带一个工程细节：删掉的东西要删干净（字段、方法、线程、配置项、`#include <thread>`），否则后人看到会以为还要用。

面试可以这么说：我们删掉过一段"信号周期模拟"代码，它按固定间隔伪造周期边界。问题是现场看到有输出就以为信号链路通了，真实链路坏了也发现不了。后来改成只认真实信号报文的相位跳变，没数据就不出数。

2-4 时间戳乱序只打了个告警

commit：`ee4907d`（2026-08-18，fengwenxi）

现象

上游推送的帧时间戳偶尔会**倒退**（后一帧的时间戳比前一帧小）。代码里只有一层判断：

```
if ((detected_result.timestamp > _current_period_start) &&
    (detected_result.timestamp - _current_period_start) >= _ensemble_detect_period) {
    if (detected_result.timestamp - _current_period_start - _ensemble_detect_period > _ensemble_detect_period) {
        std::cerr << "Warning, detected_result timestamp: " << ... << " is much later than current period start";
    }
    // 结算
} else {
    std::cerr << "Warning! detected_result.timestamp: " << detected_result.timestamp
              << ",_current_period_start: " << _current_period_start
              << ",_ensemble_detect_period: " << _ensemble_detect_period << std::endl;
}
```

这条改了什么

其实**没修逻辑**，只是把 `else` 分支加上了（原来乱序时**什么都不打印**，静默丢帧）。改动本身很小，但揭示了一个态度问题：

> 上游乱序 → 只告警、不重排、不丢弃标记。数据被静默跳过。

能学到什么

- **时间戳乱序是流式处理系统的必答题**，标准解法是**水位线（watermark）+ 允许迟到窗口**：维护"当前已见过的最大时间戳"，只对早于 `max_ts - allowed_lateness` 的数据做丢弃，之间的小幅乱序靠重排序缓冲吸收。
- 只打日志的后果：**丢了多少帧、丢了哪些，事后完全查不出来。**至少应该计数（`_dropped_out_of_order_count`）并周期性上报。
- 这是这个项目里少数"知道有问题但只加了日志"的地方 —— 如果你面试被问"怎么处理乱序"，正好可以答这套，比项目本身的做法更完整。

2-5 断流后的槽合并

commit：`90982e3`（2026-09-07）

同一个 `should_output` 里，断流恢复时的处理：

```
const uint64_t merged_windows = current_time / interval_ms - window_open_ms / interval_ms;
if (merged_windows > 1) {
    std::cerr << "[OutputChannel] " << cycle_type << " merged " << merged_windows
              << " windows of " << interval_ms << "ms, no frame in between" << std::endl;
}
```

设计取舍：跨过的槽**不补发空报文**，因为"没有帧就没有数据"， 补发一堆 0 反而会让下游以为"那几分钟真的没车"。

代价是：下游看到的两条报文时间戳差了好几倍周期，必须能容忍。 所以打 `merged N windows` 告警，让运维知道是断流，不是模块卡死。

能学到什么：断流恢复的处理只有三种选择 —— 补空、合并、丢弃。 选哪个取决于"下游怎么理解缺数据"。这里的判断是：**合并 + 告警**， 因为"0 流量"和"没观测"是两件事（和 1-7 的哨兵问题正好呼应）。

第三部分：状态与并发

这一类最像"面试题"，但每一条都是这个项目真实踩过的。

3-1 条件变量的谓词写反了

commit：`a818aa8`（2024-10-17，fengwenxi，修 `PopFront` / `PopBack` 的超时重载）

原来的写法

```
bool PopFront(T& item, const int& timeout) {
    std::unique_lock<std::mutex> lock(mutex_);
    const auto& is_timeout =
        consumer_.wait_for(lock, std::chrono::microseconds(timeout),
                           [&] { return !active_ || deque_.empty(); });   // ← 谓词
    ...
    item = deque_.front();      // 从队列取
}
```

为什么错

`wait_for(lock, rel_time, pred)` 的第三个参数语义是：**`pred` 返回 `true` 就停止等待**（代表"条件已满足"）。

而这里的谓词写的是 `deque_.empty()` —— **队列空就停止等待。**两个后果：

1. 队列**空**时，wait 立刻返回 → 接着 `deque_.front()` 取空队列 → **未定义行为**（通常崩或读到脏数据）。
2. 队列**有数据**时，谓词返回 false → 反而一直等到超时才返回 → 超时后 `return false`， **明明有数据却报超时。**

正确的谓词应该是"**有数据**或**要退出**才停"：

```
[&] { return (!active_) || (!deque_.empty()); }
```

同一提交里 `PopBack` 的超时重载也是同样的错（`deque_.empty()`）。

能学到什么

- **条件变量的谓词，永远回答"我现在可以继续了吗"**，不能描述"我现在在等什么"。 这是 `wait` 系列最容易写反的地方，而且**编译器不报错、单线程测试也不容易发现** （因为单线程里往往是先 push 再 pop，队列非空，谓词恒 false → 一直等到超时， 表现成"偶发超时"，很像性能问题）。
- 一个自检口诀：**谓词 = 退出条件**（`!active_`）**+ 业务条件**（`!queue.empty()` / `size < cap`）。 两个都用"可以继续"的形式，不要用"不满足"的形式。
- 更安全的写法是把两个重载合并成一个（见 6-1），**减少复制粘贴的机会**—— 这个 bug 就是同一个谓词在 4 个函数里各抄一遍、抄错两遍。

> **面试可以这么说**： `wait` 的谓词要表达"条件已满足"，不是"我在等什么"。 我见过把 `queue.empty()` 写进谓词的，结果是队列空时立刻返回、从空队列取值， 队列有数据时反而一直等。这类 bug 多线程下才现形，看起来像偶发超时。

3-2 析构时消费者永远卡在 `wait` 上

commit：`6b7c6af`（2024-10-15，fengwenxi）

原来的写法

```
bool PopFront(T& item) {
    std::unique_lock<std::mutex> lock(mutex_);
    while (deque_.empty()) {
        consumer_.wait(lock);          // 无谓词，被唤醒后回到 while 条件重新判断
    }
    if (active_) {
        item = deque_.front();
        deque_.pop_front();
        return true;
    } else {
        return false;
    }
}
```

析构函数：

```
~BlockingQueue() {
    {
        std::lock_guard<std::mutex> lock(mutex_);
        active_ = false;
    }
    consumer_.notify_all();
    if (thread_.joinable()) thread_.join();     // 等消费者退出
}
```

为什么错

析构时：`active_ = false` → `notify_all()` 唤醒消费者 → 消费者从 `wait` 返回 → **回到 `while (deque_.empty())` 重新判断** → 队列还是空的 → **继续 wait。**

`while` 的条件只看 `deque_.empty()`，**根本不看 `active_`**。 所以消费者永远醒不过来，`join()` 永久阻塞，**整个进程退出时卡死。**

只有在"析构时队列恰好非空"的情况下才能退出——所以这个 bug 在测试里经常"跑得通"。

改法

```
while (deque_.empty()) {
    consumer_.wait(lock);
    if (!active_) {          // 被唤醒后先看是不是要退出
        return false;
    }
}
```

四个 pop 函数（`PopFront` / `PopFront+timeout` / `PopBack` / `PopBack+timeout`）全部改。 后来的版本（`a818aa8`）进一步改成谓词形式：

```
consumer_.wait(lock, [&] { return (!active_) || (!deque_.empty()); });
if ((!active_) || (is_timeout == std::cv_status::timeout)) {
    return false;
}
```

能学到什么

- `while (条件) wait()` **这种无谓词写法，必须把"退出标志"写进** `while` **条件**， 否则就是这条 bug：唤醒了但条件不变，等于没唤醒。
- 这个 bug 的可怕之处：**只在"退出时队列为空"时触发。** 压力测试跑一晚上都发现不了，线上一次正常退出就卡住。
- 判断一个阻塞队列实现靠不靠谱，只看一点：**析构时队列为空，** `join` **能不能返回。**

> **面试可以这么说**：阻塞队列的退出是经典坑。 `while(queue.empty()) wait()` 这种写法， 析构时就算 `notify_all` 唤醒了，消费者回到循环照样看到队列空、继续等， `join` 就永久阻塞。要么把退出标志放进 `while` 条件，要么用带谓词的 `wait`。

3-3 一个 `bool` 标志被多个通道抢着消费

commit：`0e6486b`（2026-09-08，fengxin）

原来的写法

```
class SignalPhaseManager {
    bool _cycle_boundary = false;

    bool check_and_consume_cycle_boundary() {       // 消费型接口
        std::lock_guard<std::mutex> lock(_mutex);
        if (_cycle_boundary) {
            _cycle_boundary = false;                // 谁读到谁清掉
            return true;
        }
        return false;
    }
};
```

```
// 每个 OutputChannel 都这样调
bool should_output(uint64_t current_time, SignalPhaseManager* spm) {
    if (cycle_type == "signal_cycle") {
        if (!spm || !spm->check_and_consume_cycle_boundary()) return false;
        ...
    }
}
```

为什么错

配置里可能同时存在**多个** `signal_cycle` **通道**（比如一个输出车道级、一个输出流向级）。 它们共用一个 `SignalPhaseManager`。

真实周期边界来了 → `_cycle_boundary = true` → 通道 A 先调到， 读走 `true` 并清成 `false` → **通道 B 再调就读不到了**，这个周期 B 直接漏报。

谁先谁后取决于 `_output_channels` 的遍历顺序，所以表现为 "**一个周期的数据有，下一个周期是另一个通道有**"——极难查。

改法：从"标志位"改成"计数器 + 各自水位"

```
class SignalPhaseManager {
    uint64_t _cycle_boundary_count = 0;
    uint64_t cycle_boundary_count() const {      // 只读，不消费
        std::lock_guard<std::mutex> lock(_mutex);
        return _cycle_boundary_count;
    }
};
```

```
struct OutputChannel {
    uint64_t seen_cycle_boundary = 0;           // 每个通道记自己看到第几次

    bool check_trigger(uint64_t current_time, SignalPhaseManager* spm) {
        if (is_signal) {
            const uint64_t boundary_count = spm ? spm->cycle_boundary_count() : 0;
            if (boundary_count == seen_cycle_boundary) {
                return false;                   // 没新边界
            }
            seen_cycle_boundary = boundary_count;
        }
        ...
    }
};
```

注释写得很清楚：

> 参考相位「非绿 → 绿」的次数，即已到达的信号周期边界数。各通道记住自己看到过的次数， 因此一条信号报文可以同时触发多个 signal_cycle 通道。

能学到什么

- **"消费型"接口天生只能有一个消费者。**一旦可能有多个观察者，就必须改成 **"广播（计数/版本号）+ 各自记住水位"**。这是 pub-sub 的核心，也是消息队列 offset 的思路。
- 反例：`check_and_consume`、`take()`、`pop()`、`clear_flag()` 这类命名， 出现在"可能被多人调用"的地方，就是信号。
- 更进一步：计数器会溢出吗？`uint64_t` 每秒一次也要几十亿年，可以不管。 但如果改成 `uint32_t` 就要考虑回绕，`==` 判断会失效。

> **面试可以这么说**：我们原来用一个 bool 表示"信号周期到了"，谁读到谁清掉。 后来有两个通道都要用这个边界，就出现"一个周期只有一个通道出数"。 改成计数器以后，每个通道记自己看到第几次，一条信号可以触发多个通道。

3-4 用合法值 `0` 当"还没观测到"

commit：`0e6486b`（2026-09-08，fengxin）

原来的写法

```
int _prev_reference_light = 0;      // 用 0 表示"还没有历史状态"

// 判断"是不是从非绿变绿"
bool now_green = (light == LIGHT_PERMISSIVE_GREEN || light == LIGHT_PROTECTED_GREEN);
bool was_green = (_prev_reference_light == LIGHT_PERMISSIVE_GREEN ||
                  _prev_reference_light == LIGHT_PROTECTED_GREEN);
if (now_green && !was_green && _prev_reference_light != 0) {    // 0 当"未观测"
    _cycle_boundary = true;
}
```

为什么错

信号灯枚举里 `0` **是合法状态**（灭灯/未点亮）：

```
constexpr int LIGHT_XXX = 0;    // 灭灯是真实存在的状态
constexpr int LIGHT_YELLOW = 7;
```

于是两种情况无法区分：

1. **还没收到过任何信号报文**（真的未观测）→ 不该触发周期边界，因为不知道之前是什么灯。
2. **之前是灭灯，现在变绿**（真实的灭灯→绿）→ 这**应该**触发。

原代码用 `_prev_reference_light != 0` 排除第 1 种，结果把第 2 种也排掉了 —— **每次从灭灯转绿都会漏一个周期。**

改法：用独立的布尔表达"观测过"

```
int _prev_reference_light = 0;
bool _reference_observed = false;      // light=0 是合法状态，不能用它当"未观测"的哨兵

// ...
// 首个观测只记状态：无法判断它是不是刚转绿的，贸然触发会上报一个不完整的周期。
if (_reference_observed && now_green && !was_green) {
    ++_cycle_boundary_count;
    std::cout << "[SignalPhase] Cycle boundary TRIGGERED, count=" << _cycle_boundary_count << std::endl;
}
_prev_reference_light = light;
_reference_observed = true;
```

`set_reference_phase_id()` 里也要复位：

```
void SignalPhaseManager::set_reference_phase_id(int phase_id) {
    std::lock_guard<std::mutex> lock(_mutex);
    _reference_phase_id = phase_id;
    _prev_reference_light = 0;
    _reference_observed = false;       // 换相位后重新开始观测
}
```

能学到什么

- **不要用业务合法值当"未初始化"标记。**这是 C 风格代码最常见的缺陷之一 （`-1`、`0`、`nullptr`、空串都可能合法）。
- 正确姿势：**额外的状态位**（`observed` / `has_value` / `std::optional`）。 C++17 有 `std::optional<T>` 就是干这个的。
- 这条和 1-7 的哨兵之争是同一类问题的两个侧面： 1-7 是"数值要不要承载'没有'"的取舍，3-4 是"数值承载了会出什么错"。

> **面试可以这么说**：我们信号周期漏过周期，原因是拿 `light == 0` 当"还没收到信号"的标记， 但 0 本身就是灭灯这个合法状态，所以每次灭灯转绿都被当成首次观测、不触发。 后来加了一个 `observed` 布尔。这类问题我的原则是不用业务值当哨兵，要么用 `optional`，要么单开一个状态位。

3-5 追踪表只增不删 —— 跑一天内存涨到几个 G

commit：`90982e3`（2026-09-07，fengxin）

原来的写法

```
class StopLineCrossingTracker {
    std::map<std::string, math::Vec2D> _prev_positions;    // 目标 ID → 上一帧位置
    std::set<std::string> _crossed_ids;                    // 已过线的 ID（去重）
};
```

```
for (const auto& obj : frame.traffic_objects) {
    ...
    _prev_positions[obj.id] = current_pos;      // 只写，永不删
}
```

为什么错

过线判定需要"上一帧位置"，所以每个出现过的目标都要存一条。 但**目标离开视野后再也不会被访问，条目却永远留着。**

一个路口一天几万个目标 ID，每条 `std::string` + `Vec2D` 几十字节， 加上 `std::map` 的红黑树节点开销 → 一天几百 MB，跑一周就 OOM。

`_crossed_ids` 同样只增不删，而且它在 `calculate_once()` 里被 `clear_crossing_state()` 清掉， 反而 `_prev_positions` 没人清。

改法：加 TTL，每帧开头清一遍

```
struct PrevState {
    math::Vec2D position;
    uint64_t last_seen = 0;         // 上次见到这个目标的帧时间戳
};

std::map<std::string, PrevState> _prev_positions;
```

```
// process_frame 开头
for (auto it = _prev_positions.begin(); it != _prev_positions.end();) {
    if (frame.timestamp > it->second.last_seen && frame.timestamp - it->second.last_seen > POSITION_TTL) {
        it = _prev_positions.erase(it);        // erase 返回下一个迭代器，不能 it++
    } else {
        ++it;
    }
}

// 循环里更新
_prev_positions[obj.id] = {current_pos, frame.timestamp};
```

```
constexpr uint64_t POSITION_TTL = 10000;        // 10 秒没再出现就清掉
```

能学到什么

- **任何"按实体 ID 建的缓存"，都必须有清理策略。**三种常见做法： ① 帧内用完就清（最简单，但跨帧状态丢了）； ② TTL / LRU（本文做法）； ③ 固定容量环形缓冲（`afl::BlockingQueue` 就是这种，满了丢最老）。
- 细节：`erase` **之后迭代器失效**，必须用返回值：

  ```
  for (auto it = m.begin(); it != m.end();) {
      if (要删) it = m.erase(it); else ++it;
  }
  ```

  写成 `m.erase(it); ++it;` 是未定义行为。
- 另一个细节：`frame.timestamp > last_seen` 这个判断是为了防**时间戳倒退**时算出巨大的差值 （`unsigned` 相减会回绕成天文数字，把所有条目一次清空）。这一手很细。

3-6 「没数据」被当成「没这个相位」

commit：`500bc4a`（2026-09-09，fengxin）

背景

信号相位查询接口要回答"某个相位的灯是不是绿的"。这里有个三值问题：

情况	正确答案
有这个相位，当前绿灯	放行
有这个相位，当前红灯	不放行
有这个相位，但**还没收到任何灯色**	**不知道**
这个路口根本没有这个相位	不放行 / 视为不受控

原来的实现把 3 和 4 混了：只要 `_phase_states` 里查不到，就返回"不是绿灯"。 而调用方（比如某些指标）把"不是绿灯"当成"不受信号控制，可以算"。

注释写得很清楚

```
// 一条有效灯色都没缓存过 ⇒ 信号链路本身没数据。这时不能按「本路口没这个相位」解释，
// 否则等于给全路口的相位都判成不受控放行。
```

改法

`on_signal_data` 里改成**所有相位都留档**，而不是只缓存参考相位：

```
const int light = state["light"];
// 全相位都留档：目前只有周期切分用参考相位，绿信比/WasteTime 等后续指标要查任意相位。
_phase_states[phase_id] = light;

if (phase_id != _reference_phase_id) {
    continue;
}
// 只有参考相位才进一步判"非绿转绿"
```

这样"查不到这个相位"就只剩**一种**含义（报文里确实没这个相位）， 而"整个 map 是空的"才是"信号链路没数据"——两者可以区分开。

能学到什么

- **"缺失"要有层次**：没收到数据 / 收到了但没有这一项 / 有这一项但值未知。 三值逻辑（true/false/unknown）在告警、权限、信号控制里到处都是， 用 bool 硬表达一定会出这种错。
- 判断顺序很重要：**先判"链路有没有数据"，再判"这一项在不在"**。
- 这也是"默认值安全"问题：任何 `return false` 的兜底， 都要问一句"调用方会不会把 false 当成'允许'"。

第四部分：协议与输出

对接方读不到数据时，八成是这一类的锅。

4-1 `LaneNO` 写成 `LaneNo` —— 一个字母的事

commit：`7a5928c`（2025-06-26，fengwenxi）

改动

```
-    coil_result["LaneNO"] = coil_info.lane_no;
+    coil_result["LaneNo"] = coil_info.lane_no;
```

一个字符（O 的大小写）。

为什么影响大

JSON 的 key **大小写敏感**。协议文档里写的是 `LaneNo`， 下游按 `LaneNo` 取值 → 取不到 → 拿到默认值 / 空 → 那条车道的标识全空。

而且这种错**不会报任何异常**：JSON 是合法的，只是 key 不匹配。 排查时看上游日志"数据都正常输出了"，看下游"字段是空的"，中间卡很久。

能学到什么

- **JSON/C++ 结构体映射的字段名，必须有一处唯一真值。** 手工写字符串 `["LaneNo"]` 是没有任何编译期保护的。
- 后面 fengxin 的做法说明了正确方向（`90982e3`）—— 用 `constexpr std::array<const char*, 5> kVolumeDetailKeys = {"Volume1",...}`， 把字段名集中成常量数组，循环使用，**拼错一次全局只有一个地方要改。**
- 再进一步是代码生成（protobuf / 从协议文档生成结构体），但这个项目没做。

4-2 该给真值，却给了内部 tag

commit：`90982e3` → 修正于 `1f4b516`（2026-09-16，fengxin）

原来的写法

```
// 分支级输出
entry["NodeID"]          = branch.value("road_section_tag", "");    // ← 内部 tag
entry["UpstreamCrossID"] = branch.value("road_section_tag", "");    // ← 同一个 tag
entry["BranchID"]        = branch.value("road_section_tag", "");    // ← 还是 tag
```

三个字段全填了同一个**内部断面 tag**（形如 `"2_ENTRY"`）。

为什么错

`NodeID` / `UpstreamCrossID` 是**路口编号**（外部协议字段），`road_section_tag` 是代码内部拼出来的断面标识。两者语义完全不同，下游拿 `"2_ENTRY"` 当路口编号去匹配，永远匹配不上。

改法：按 tag 反查 ROI 配置

```
void fill_section_ids(nlohmann::json& entry,
                      const std::unordered_map<std::string, RoadSectionInfo>& road_sections,
                      const std::string& section_tag) {
    auto find_it = road_sections.find(section_tag);
    const bool found = find_it != road_sections.end();
    entry["NodeID"]          = found ? find_it->second.node_id : "";
    entry["UpstreamCrossID"] = found ? find_it->second.upstream_cross_id : "";
    entry["BranchID"]        = static_cast<int>(found ? find_it->second.direction : DirectionCode::DEFAULT);
    entry["BranchType"]      = (found && find_it->second.type == LaneType::EXIT_SECTION) ? 1 : 0;
}
```

注释里还解释了为什么 **BranchID 要用 8 方向码而不是 tag**：

> 记录标识里的 BranchID 上报断面的 8 方向码，即 ROI 的 f_dir_8_no 原值（1北 2东 3南 4西 5东北 6东南 7西南 8西北）

以及为什么 **BranchType 必须三个颗粒度都带**：

> 同方向的进口道/出口道/人行横道共用一个方向码，只靠 BranchID 分不开断面，下游要靠 BranchID+BranchType(+FlowType/LaneID) 才能唯一定位记录。

能学到什么

- **"内部标识"和"外部标识"必须是两套东西**，输出前必须查表翻译。直接把内部 ID 塞进协议字段，是"看起来有值、实际对不上"的典型。
- **唯一性要论证**：一个字段能不能唯一定位一条记录，要想清楚。这里 `BranchID`（方向码）就做不到，因为同方向有进口道/出口道/人行横道三条断面，所以必须加 `BranchType`。
- 反查失败时**填空串/0 而不是猜**：`宁可显式无值，不猜方向`。

4-3 硬编码通道名映射周期值

commit：`90982e3`（2026-09-07，fengxin）

原来的写法

```
int get_cycle_value() const {
    if (cycle_type == "snapshot") return 1;
    if (cycle_type == "accumulate_5min") return 300;
    return -1;
}
```

为什么错

周期值应该由**配置里的 interval** 决定，而不是由**通道名字**决定。 一旦新增一个 "accumulate_1s" 通道，这里返回 -1（协议里 -1 表示"信号周期级"）， 下游会把 1 秒级报文当成信号周期级解析。

而且改配置不改名字时， `Cycle` 字段就和实际周期对不上。

改法

```
int get_cycle_value() const {
    // signal_cycle 的窗口由相位边界决定，不是固定秒数；其余通道节奏就是 interval_ms。
    if (cycle_type == "signal_cycle") return -1;
    return static_cast<int>(interval_ms / 1000);
}
```

只保留一个"真的特殊"的分支（信号周期级用 -1 标记），其余全部从数据推导。

能学到什么

- **凡是 `if (name == "xxx") return 常量` 的写法，都要问一句"这个名字会变吗"。** 名字是给人看的，行为应该由参数决定。
- 反过来， `signal_cycle` 用 `-1` 是合理的特殊值，因为它的周期确实不是固定秒数 —— 这种"真的没有固定值"才能用哨兵。

4-4 `spatial_output` 挂错了层级

commit： `90982e3` （2026-09-07）

原来的配置结构

```
{
  "cycle_type": "accumulate_5min",
  "detectors": [
    { "type": "volume", "target": "entry_lane", "spatial_output": ["FLOW", "LANE"] },
    { "type": "queue",  "target": "entry_lane", "spatial_output": ["LANE"] }
  ]
}
```

`spatial_output` 挂在**每个检测器**下面。

为什么有问题

`spatial_output` 决定的是**一条报文包含哪些空间颗粒度**（FLOW / LANE / BRANCH）， 而报文是**通道级**的——一个通道出一条报文。挂在检测器上会导致：

- 多个检测器的 `spatial_output` 要取并集，代码里就是 `channel.spatial_output.insert(so)` 堆在一起；
- 同一个通道里两个检测器配置不一致时，语义说不清（volume 要 FLOW，queue 不要，那这条报文到底有没有 FLOW？）。

改法

配置上移到通道级：

```
{
  "cycle_type": "accumulate_5min",
  "spatial_output": ["FLOW", "LANE"],
  "detectors": [ { "type": "volume" }, { "type": "queue" } ]
}
```

代码里加了校验：

```
for (const auto& spatial : cycle_config.spatial_output) {
    if (kValidSpatialOutputs.count(spatial) == 0) {
        std::cerr << "[BuildDetectors] Channel [" << channel.cycle_type
                  << "] has invalid spatial_output '" << spatial << "', ignored" << std::endl;
        continue;
    }
    channel.spatial_output.insert(spatial);
}
if (channel.spatial_output.empty()) {
    std::cerr << "[BuildDetectors] Channel [" << channel.cycle_type
              << "] has empty spatial_output, its reports will carry no Coil_List entry" << std::endl;
}
```

同时 `DetectorItemConfig` 瘦身成只剩 `type`：

```
// 检测器项只声明指标类型；其 ROI 域由 type 隐含决定（volume/queue/speed→进口道线圈，crosswalk→斑马线）。
struct DetectorItemConfig {
    std::string type;
};
```

—— 原来的 `target` 字段也被删了，因为 `type` 已经隐含了 ROI 域。

能学到什么

- **配置项该挂在"它真正属于的那一层"**。判断方法：**改变这个值，影响的是谁？** 影响整条报文 → 挂通道；只影响某个检测器 → 挂检测器。
- 配置瘦身和代码瘦身一样重要：**能由其它字段推导的字段，不要出现在配置里**（`target` 就是）。
- 空配置要告警：`spatial_output` 为空会导致报文里没有 `Coil_List`， 这是"合法但肯定不是本意"的配置，必须报出来。

4-5 `-ffast-math` 让 `round(v*100)/100` 失效

commit：`90982e3` 里的注释（2026-09-07，fengxin）

背景

报文里浮点字段要统一保留 2 位小数。naive 写法：

```
value = std::round(v * 100.0) / 100.0;
```

为什么不行

项目 `CMakeLists.txt` 的生产构建开了 `-O3 -ffast-math`（第 90 行左右）。 `-ffast-math` 会打开 `-freciprocal-math`，GCC 据此把「除以常量 100.0」优化成「乘 0.01」。

而 **0.01 在二进制里不可精确表示**，`v * 0.01` 和 `v / 100.0` 结果可能差 1 ulp， `std::round` 在边界上就会进位错误 —— 该是 `1.80` 的可能变成 `1.79`。

实际做法

```
// 报文浮点字段统一保留 2 位小数。nlohmann 对 double 走「最短往返」序列化，没有按位格式化的开关，
// 所以只能先把值舍到 0.01 再写：这同时消掉 float 累加带出的假精度尾数（1.7999999523162842 -> 1.8）。
// 只碰 number_float，整型字段（计数、标志位、Cycle 等）原样出。
//
// 必须走「格式化成 2 位小数字符串再读回」，不能写 std::round(v * 100.0) / 100.0：生产构建开了
// -ffast-math（CMakeLists.txt:90），GCC 据此把「除以常量 100.0」换成「乘 0.01」，而 0.01 二进制不可...
```

也就是**不走算术，走字符串**： `snprintf("%.2f")` 或 `to_string` 再 `stod` 读回， 格式化是**精确的十进制舍入**，不受浮点指令重排影响。

能学到什么

- `-ffast-math` **会让浮点结果不符合 IEEE**：重排结合律、把除法变乘法、假设无 NaN/Inf。 用它就要接受"数值不可复现"。
- 涉及**金额、坐标、协议输出**的浮点运算，**不能用** `-ffast-math`。 真要性能就分文件编译（`set_source_files_properties(... COMPILE_OPTIONS -fno-fast-math)`）。
- 十进制舍入的正确姿势是**字符串格式化**，不是乘除取整。这条在 C++ 里尤其反直觉。
- 附带学到： `float` 累加会带出假精度（`1.7999999523162842`）， 输出前统一舍到 2 位能顺手把这类噪声消掉。

> **面试可以这么说**： `-ffast-math` 下 `round(v*100)/100` 不可靠， 因为编译器会把除以 100 换成乘 0.01，而 0.01 二进制不精确，边界上会差一个 ulp。 协议输出这种要求精确十进制的地方，得走格式化字符串再读回。

第五部分：工程与构建

比前几类"笨"，但最容易在换环境时炸。

5-1 漏 `#include <limits>` —— 换个编译器就炸

commit：`5219596`（2025-05-28，lijin1）

```
 #include <algorithm>
+#include <limits>

 namespace fusion_perception::math {
 const double DOUBLE_EPSILON = std::numeric_limits<double>::epsilon();
```

为什么能编过

`<limits>` 很可能被别的头文件**间接包含**了（`<algorithm>`、`<cmath>`、某个第三方头）。 在 GCC 某个版本上能过，换个编译器/标准库版本就报 `'numeric_limits' is not a member of 'std'`。

能学到什么

- **"在我机器上能编过"不等于代码正确。**头文件依赖必须显式写全。
- 这类错最典型的表现是：**CI 上换个镜像就编译失败，本地怎么都不复现。**
- 工程手段：`include-what-you-use`（IWYU）工具能自动检查； 或者用 `-Werror` + 严格的编译选项组合跑一遍。

5-2 图片文件名按字符串排序 —— `img10` 排在 `img2` 前面

commit：`4f73eaa`（2025-10-21，fengwenxi）

原来的写法

```
if isinstance(image_dir_or_files, list):
    self.image_files = image_dir_or_files        # 直接用传进来的顺序
```

为什么错

调用方传进来的列表通常来自 `os.listdir()` / `glob()`，顺序是**文件系统的顺序**（近似随机）； 即使排序，默认的字符串排序也是：

```
img1.jpg, img10.jpg, img2.jpg, img3.jpg ...
```

—— `"10" < "2"` 按字符串成立。结果**抽帧回放时画面顺序乱跳。**

改法：自然排序（natural sort）

```
import re

if isinstance(image_dir_or_files, list):
    # natural sort by basename with numeric awareness, case-insensitive
    def natural_key(path):
        name = os.path.basename(path)
        # split into digit / non-digit parts, convert digit parts to int for numeric ordering
        return [int(t) if t.isdigit() else t.lower() for t in re.split(r'(\d+)', name)]
    # sort the incoming list so order is deterministic (and numbers sort naturally)
    self.image_files = sorted(image_dir_or_files, key=natural_key)
```

`re.split(r'(\d+)', name)` 把 `"img10.jpg"` 切成 `['img', '10', '.jpg']`， 数字段转 `int` → 比较时 `10` 和 `2` 按数值比 → 顺序正确。

能学到什么

- **"文件名排序"几乎从来不是字符串排序。**凡是有编号的文件（帧、分片、版本、日志）， 都要用自然排序。
- `key` 函数混合返回 `int` 和 `str` 会不会报错？**在 Python 里会**（`TypeError`）， 但这里的 `re.split` 保证切出段落的**奇偶位置类型一致**（偶数位是文本，奇数位是数字）， 所以两个列表比较时同一位置永远同类型。这个技巧值得记。
- 附带：这个改动还顺手把"列表输入"也做了排序，之前只有"目录输入"那条分支排了序 —— **两条分支行为不一致**是很常见的疏漏。

5-3 `CMAKE_MINIMUM_REQUIRED(VERSION 3.15)` 定太高

commit：`ee4907d`（2026-08-18，fengwenxi）

```
-CMAKE_MINIMUM_REQUIRED(VERSION 3.15)
+CMAKE_MINIMUM_REQUIRED(VERSION 3.0)
```

为什么

目标设备（Ubuntu 20.04 / JetPack）自带的是 **CMake 3.16**，某些老容器里甚至只有 3.10/3.13。 写 3.15 会把一批环境挡在门外，而代码其实只用到 3.0 的语法。

同一次提交里还有：

```
-#DIRECTORY(GLOB CONFIG_FILES ${CMAKE_CURRENT_SOURCE_DIR}/config/**)
-INSTALL(DIRECTORY ${CMAKE_CURRENT_SOURCE_DIR}/config/ DESTINATION ${CMAKE_INSTALL_CONFIGDIR})
+FILE(GLOB CONFIG_FILES ${CMAKE_CURRENT_SOURCE_DIR}/config/*)
+INSTALL(FILES ${CONFIG_FILES} DESTINATION ${CMAKE_INSTALL_CONFIGDIR})
```

这两句的区别

- `INSTALL(DIRECTORY ...)`：**按目录安装。**源目录里有子目录、隐藏文件、空目录都会被带进去； 目标路径会**多一层目录名。**
- `FILE(GLOB)` + `INSTALL(FILES ...)`：**只装文件**，路径干净，且可以显式控制层级。

踩过的坑通常是：`INSTALL(DIRECTORY config/ DESTINATION xxx)` 会把 `config/` 下面的 **子目录结构整体复制**，而部署脚本期望的是"所有 json 平铺在 xxx 下" → 程序启动找不到配置。

> **能学到什么**

- `cmake_minimum_required` **写"你真正需要的最低版本"，不是"你本机的版本"。** 写高了纯粹给自己添堵，尤其要交付到别人环境。
- `FILE(GLOB)` 有个众所周知的缺点：**新增文件不会触发重新配置**（需要手动 `cmake .`）。 生产级做法是显式列出文件。但对"配置/文档"这种随包走的资源，`GLOB` 是可以接受的。
- `INSTALL(DIRECTORY)` vs `INSTALL(FILES)` 的差别，只有在**部署后路径不对**时才被发现 —— 所以打包流程一定要在干净容器里验一遍。

5-4 顺手修的一堆小东西（同批提交里的）

这些不值得单独成篇，但都是"真实工程里天天遇到的"：

位置	问题	改法
`CMAKE_MINIMUM_REQUIRED`	3.15 挡住老环境	降到 3.0
`Dockerfile`	`WORKDIR /work/fusion_perception` 与部署路径不一致	统一成 `/work/cictci/perception`
`Dockerfile`	没设时区 → license 校验失败	`ENV TZ=Asia/Shanghai` + `tzdata`（注释写明了原因）
`docker-compose.yaml`	volume 挂载路径跟着 WORKDIR 一起错	同步改
`.gitignore`	测试数据被提交进仓库	加 `**/test/`
`install(DIRECTORY ...)`	装出一堆子目录	改 `GLOB + FILES`

能学到什么：`Dockerfile` 里"为什么要设时区"这种注释， **不写下来三个月后一定被删掉**。他写的是：

```
# 设置时区,不然license验证失败
ENV TZ=Asia/Shanghai
```

—— 一句话说明了"看起来无关但必需"的原因，这种注释的价值极高。

第六部分：不是 bug，但决定了后面出多少 bug

6-1 四个重载各抄一遍 —— 那份复制粘贴里藏了两个 bug

commit：`a818aa8`（2024-10-17，fengwenxi），净减 148 行

原来有 8 个 push/pop 重载

```
void PushBack(const T& item);
void PushBack(T&& item);
void PushBack(const T& item, const int64_t& timeout);
void PushBack(T&& item, const uint64_t& timeout);
void PushFront(const T& item);
void PushFront(T&& item);
void PushFront(const T& item, const uint64_t& timeout);
void PushFront(T&& item, const uint64_t& timeout);
bool PopFront(T& item);
bool PopFront(T& item, const int& timeout);
bool PopBack(T& item);
bool PopBack(T& item, const int& timeout);
```

`wait` 的谓词、容量判断、`active_` 判断在**每个函数里各写一遍**。

结果

3-1 里那个"谓词写反"的 bug，出现在 `PopFront(T&, timeout)` **和** `PopBack(T&, timeout)` 里 —— 因为是从同一个函数复制过去改的，**改的时候只改了一半**。

另外 `PushFront(const T& item)` 改成 `bool` 返回值后，**函数体里根本没写** `return` —— 这个缺陷一直留到后面版本都没修（返回值是未定义行为）。

改法

合并成"一个实现 + 默认参数"：

```
bool PushBack(const T& element, const int64_t& timeout = 0);   // timeout=0 表示一直等
bool PushBack(T&& element, const uint64_t& timeout = 0);
bool PushFront(const T& element, const uint64_t& timeout = 0);
bool PopFront(T& element, const int& timeout = 0);
bool PopBack(T& element, const int& timeout = 0);
```

12 个函数降到 8 个（`const T&` 和 `T&&` 仍需分开，因为要保留移动语义），**每个逻辑只有一份**。

能学到什么

- **重复代码的代价不是"行数多"，而是"改的时候会漏"**。这里漏出了一个谓词反了、一个缺 return。
- C++ 里"完美转发"能把 `const T&` 和 `T&&` 合成一个模板函数，但他没这么做（可能为了让头文件更易读）—— 这是合理的取舍：**两个重载 + 一个实现**比"模板 + `enable_if`"更好读。

- 反过来看：写这种基础库时，**第一个要问的是"能不能只写一份"**， 第二个是"测试怎么覆盖到每一份"。这个项目没有单元测试，所以两份都错了也没人发现。

6-2 `ExitLaneInfo` 瘦身 —— 删掉"没人读"的字段

commit：`0e6486b`（2026-09-08，fengxin）

```
 struct ExitLaneInfo {
     std::string tag;
-    std::string lane_id;
-    std::string lane_no;
-    int32_t coil_no;
-    std::string turn_dir_no;
     RoadSectionInfo road_section_info;
-    std::vector<math::Vec2D> contour_segment_1;
-    std::vector<math::Vec2D> contour_segment_2;
-    std::vector<math::Vec2D> stop_line_points;
-    std::vector<math::Segment2D> stop_line_segments;
-    math::Polygon2D lane_contour;
     math::Polygon2D overflow_zone_near;
     math::Polygon2D overflow_zone_far;
     math::Polygon2D overflow_zone_upstream;
 };
```

注释：

```
// 出口道只被溢流检测器读取：断面标识 + 三级溢流区域。
// 出口道不需要车道轮廓/停止线（只有进口道线圈有读者），因此配置里的 contour 已删除。
```

为什么能删：出口道只用于溢流检测，而溢流检测只需要"三级溢流区域多边形"， 不需要车道轮廓、停止线、线圈编号。之前是**照抄进口道的结构体**，多出来的字段一直没人用， 还导致配置里必须画出口道轮廓（现场配错就崩）。

能学到什么

- **"没人读的字段"不是无害的**：它会让配置变复杂、让解析代码有分支、让新人以为要用。
- 判断方法很朴素：`grep` **一下这个字段名，看有几处读。**只有写的、没有读的，就是死字段。
- 这条改动顺带修了一个隐患：原来 `from_json` 里用

  ```
  if (!exit_lane_info.contour_segment_1.empty() && !exit_lane_info.contour_segment_2.empty())
  ```

  保护"出口道 ROI 只画溢流区域不画轮廓"的情况，注释写着 `避免空 vector 下标越界`。**字段删了，这个下标越界的风险也一起没了。**

6-3 两套管道并存 → 合成一套

commit：`90982e3`（2026-09-07，fengxin）核心重构

之前有两套

旧管道	新管道
`TrafficFlowDetector`（抽象基类）	`AccumulateDetector`（抽象基类）
`VolumeDetector` / `QueueDetector` / `HeadwayDetector` / `OccupancyDetector`	`AccumulateVolumeDetector` / `AccumulateQueueDetector` / …
`DetectorConfig` + `GlobalConfig::construct_detector_config()`	通道配置 + `build_detectors_from_config()`
`_coil_detectors`（按线圈组织）	`_output_channels`（按输出通道组织）
`create_detector()` 工厂 switch	直接 `make_unique`

两套同时在跑：`detect()` 里先遍历旧管道更新一遍，再遍历新通道更新一遍 —— **同一个目标被两套逻辑各算一次**，结果只取新的那套，旧的是纯浪费。

改法

删掉旧管道全部内容：

```
-    std::unordered_map<std::string, std::vector<std::unique_ptr<TrafficFlowDetector>>> _coil_detectors;
-    static TrafficFlowDetector* create_detector(const DetectorConfig& detector_config);
-    std::unordered_map<std::string, std::vector<DetectorConfig>> _detector_configs;
-    bool construct_detector_config();
-    std::vector<DetectorConfig> get_detector_config(const std::string& coil_tag);
```

`detect()` 里那段也删了：

```
-        for (const auto& one_section_detectors : _coil_detectors) {
-            for (const auto& one_detector : one_section_detectors.second) {
-                one_detector->update(detected_result);
-            }
-        }
```

能学到什么

- **重构的安全做法是"新管道并行跑一段，确认对齐后再删旧的"。** 这个项目就是这么干的（`f7591c0` 新增累积框架、重构工厂为双管道 → 两周后 `90982e3` 删旧）。 这比"一次性替换"安全得多，代价是中间两周有两套逻辑。
- **并行期的坑**：两套都 `update()`，如果有一方有副作用（写文件、改状态），就会互相干扰。 这里两套都是纯计算，所以安全。
- 删旧代码时**连带删配置字段**（`target`、检测器级 `spatial_output`、`simulate_interval_ms`）， 否则配置里留着"看起来还有用"的键。

第七部分：其余 fix 速览（未逐行展开的几条）

这几条改动量大、但教学价值集中在一点，所以只摘关键片段。

7-1 Python 工具里的两个 Windows 中文路径坑 —— `ebbd4d7`（2025-10-14，fengwenxi）

现场机器上工具打不开图，报 `FileNotFoundError`，路径里带中文。

坑一：`cv2.imread` 不认非 ASCII 路径。

```
# 原来
self.image = cv2.imread(image_path)          # 中文路径 → 返回 None
# 改成
self.image = cv2.imdecode(np.fromfile(image_path, dtype=np.uint8), cv2.IMREAD_COLOR)
```

`cv2.imread` 底层走的是窄字符文件 API，Windows 上遇到中文路径直接失败， 而且**不抛异常、只返回 `None`** —— 所以你会看到"文件明明在那儿，就是读不到"。 `np.fromfile` 是 Python 层实现，走 Unicode API，读成字节流再交给 `cv2.imdecode` 解码，就绕过了。

坑二：`open(..., 'r')` 不指定 encoding。

```
# 原来
with open(json_path, 'r') as f:              # Windows 默认按 GBK 解码
# 改成
with open(json_path, 'r', encoding='utf-8') as f:
```

JSON 里有中文注释或中文名字段，默认 GBK 解码就 `UnicodeDecodeError`。 **跨平台代码里，凡 `open` 必写 `encoding`** —— 这是唯一可靠的做法，别指望环境变量。

> 顺带：同一提交还加了鱼眼相机支持（`FisheyePixel2WGS84`），属于功能扩展，不是 bug。

能学到什么：Windows + 中文路径是中国现场部署的默认场景，不是边缘情况。 `cv2.imread` / `cv2.imwrite` / `open` 这三处是固定雷区，开工就该按上面的写法写。

7-2 导出 JSON 的结构和 C++ 端对不上 —— `75f6228`（2025-10-23，fengwenxi）

标注工具导出的配置文件，C++ 端解析不出来。

```
# 原来：contour 导出成对象（左边一个点、右边一个点）
"contour": {"left": left_geo, "right": right_geo},

# 改成：导出成数组（C++ 端按 std::vector<Vec2D> 读）
"contour": [left_geo, right_geo],
```

同一份配置，Python 端按字典写，C++ 端按数组读 —— 又是一次"两边口径没对齐"。 和 4-1（`LaneNO`/`LaneNo`）、4-4（`spatial_output` 挂错层级）是同一个病根： **配置 schema 没有单一权威定义，两边各写各的。**

附带一个"兜底形同虚设"的写法

同一次提交还加了导出前字段校验，但：

```
if not inter_id or fdir in ("", -1, None) or ... :
    QMessageBox.warning(self, "字段缺失", "存在车道信息字段未填写完整...")

output = {"lanes": lanes_list, "entrances": entrances}   # ← 直接往下走了
```

弹了警告框，但没有 `return`。用户看到警告点"确定"，导出照常进行，导出的还是残缺配置。 这比不校验更糟 —— 它会让人以为"工具帮我检查过了"。

> 正确写法：`QMessageBox.warning(...)` 之后 `return`（或 `return None` 让上层中断）。

能学到什么：

- 配置的 schema 一定要有一处权威定义（哪怕就是一份样例 JSON + 一个解析函数）， 两边都对照它写，否则这种"对象 vs 数组"的错永远修不完。
- **凡是"检查 + 拦截"，写完要问一句：检查失败之后，流程真的停了吗？** 这条和"规律三：兜底逻辑会掩盖真问题"是一体两面 —— 该拦截的时候没拦住，比没有检查更危险。

7-3 PCU / Volume 的配置来源统一 —— `47485fb`（2024-12-06，fengwenxi）

```
// 原来：从基类的扁平字段拿
PcuDetector::PcuDetector(const DetectorConfig& detector_config)
    : TrafficFlowDetector(detector_config),
      object_types_(detector_config.object_type),
      detect_period_(detector_config.detect_period),
      current_period_start_(0) {}

// 改成：从 custom_config_json 里取
PcuDetector::PcuDetector(const DetectorConfig& detector_config)
    : TrafficFlowDetector(detector_config), current_period_start_(0) {
    nlohmann::json custom_config_json = detector_config.custom_config_json;
    custom_config_json.at("object_type").get_to(object_types_);
    custom_config_json.at("detected_period").get_to(detect_period_);
    ...
}
```

表面是"换个地方读配置"，实质是**把"检测器自己的参数"和"框架通用参数"分开**： 基类只管框架级的东西，业务参数全塞进 `custom_config_json`。 这为后面 `90982e3` 那套"配置驱动 + 工厂统一构造"铺了路。

> 注意 `detected_period` 这个键名 —— 拼写是 `detected`（过去式）， 和别处的 `detect_period` / `ensemble_cycle` 不一致，属于历史遗留的命名债。改配置时容易写错。

能学到什么：配置收敛要趁早。这条改动本身没修 bug，但它让后面"每个检测器各读各的字段" 变成"统一从一个 json 读"，是 6-3（两套管道并存）能顺利合并的前提。

7-4 几何库合并：`geo_calc` → `geometry_calc` —— `9b476d1`（2024-12-24，

fengwenxi)

纯重构，没有 bug：删掉 `src/common/geo_calc.{h,cpp}`，能力并入 `geometry_calc`。

```
- geo_calc.h / geo_calc.cpp          （只有 IsLineIntersect 一个函数）
→ geometry_calc.h / geometry_calc.cpp（点、线、多边形、相交判定都在这）
```

一个只有单个函数的库文件，散着放不如合并 —— 找"有没有现成的相交判定"时， 没人会想到去 `geo_calc` 里翻。

能学到什么：**同一个领域的概念放在同一个文件里**，比"按函数粒度切文件"更实用。 切太碎的直接后果是重复实现 —— 后来又写了一遍相交判定，因为没人知道原来那个在哪。

规律：这个项目的错，集中在三个地方

把 20 多个案例拉通看，反复出现的只有三类：

规律一：口径没对齐（占了一半以上）

案例	口径是什么
1-1 经纬度	`Vec2D` 的 x 是纬度还是经度
1-2 车间距	中心距 / 车头距 / 车尾距
1-3 转向位序	4 位的哪一位是左转
1-4 机动车范围	自行车算不算机动车
1-5 空间占有率	密度指标还是距离指标
1-7 哨兵值	"没有样本"用 -1 还是 0
2-2 残窗	首条报文标不标完整
4-1 字段名	`LaneNo` 还是 `LaneNO`
4-2 标识	内部 tag 还是外部路口编号

共同点：这些错**都不会让程序崩溃**，只会让数字悄悄错掉。 而且都**没有单元测试兜住**。

他怎么做对的：每处修正都配一段**注释解释口径**，而且写得像"给下一个人看的说明书"。 这个项目最有价值的资产其实是这些注释，不是代码。

规律二：状态的生命周期没想清楚

案例	状态	谁负责清
2-1 相对窗口	`last_output_time`	没人（一直在漂）
3-3 周期边界	`_cycle_boundary`	第一个读者（其他人没了）
3-4 未观测	`_prev_reference_light=0`	用合法值当哨兵
3-5 追踪表	`_prev_positions`	没人（一直涨）
3-6 相位缓存	`_phase_states`	只缓存参考相位（其它查不到）

共同点：一个变量，**谁写、谁读、谁清、什么时候清**，只要有一项含糊，就出 bug。

自检清单（可以拿来审自己的代码）：

1. 这个变量是"最新值"还是"累计值"还是"事件标记"？
2. 有多个读者吗？有 → 不能用消费型接口。
3. 它的取值范围内，有没有哪个值是业务合法值（0、-1、空串）被我拿来当"没有"？
4. 它挂在某个实体 ID 上吗？→ 必须有清理策略。

规律三：兜底逻辑会掩盖真问题

案例	兜底	掩盖了什么
2-3 信号模拟	定时器伪造周期边界	真实信号链路坏了
3-6 相位查询	查不到 → 返回"不是绿灯"	"没数据"和"没这个相位"混淆
1-3 配置	`turn_dir_no: "0000"`	数据静默消失
4-2 输出	三个字段都填内部 tag	下游永远匹配不上
2-4 乱序	只打日志	丢了多少帧查不出来

他的原则（从注释里能读出来）：

> `宁可显式无值，不猜方向` `没有信号数据时本通道就是不出数，不做定时模拟` `宁可报错，也不要静默退化`

附：全部 fix 类提交清单

按时间倒序。★ = 本文详细分析过。

commit	日期	作者	内容	涉及
★ `1f4b516`	2026-09-16	fengxin	去哨兵值 -1，BranchID 改 8 方向码	factory / 4 个检测器 / config
★ `c2327c7`	2026-09-09	fengxin	左右转流向数据给反；流向级不输出排队数	spatial_aggregator / factory
★ `e868084`	2026-09-09	fengxin	行人/非机动车过街加帧数+速度双门限	crosswalk_detector / config
★ `500bc4a`	2026-09-09	fengxin	信号灯枚举与全相位缓存	signal_phase_manager
★ `997de52`	2026-09-08	fengxin	排队规则里"一个车身"的算法修正	queue_detector / config
★ `0e6486b`	2026-09-08	fengxin	信号周期去模拟化；headway 去信号灯门控；time_align	9 个文件
★ `0f2937a`	2026-09-08	fengxin	排队规则可配置 + 连续排队判定	queue/overflow detector
★ `90982e3`	2026-09-07	fengxin	周期栅格化；Volume 分 5 型；质量门控；追踪表 TTL	10 个文件
★ `ee4907d`	2026-08-18	fengwenxi	时间戳乱序告警；打包方式；CMake 降版本	factory / CMake / Docker
★ `75f6228`	2025-10-23	fengwenxi	FusionFlow 导出 json 的 bug（对象 vs 数组）	tool/*.py
★ `4f73eaa`	2025-10-21	fengwenxi	图片列表自然排序	draw_roi_page.py
★ `ebbd4d7`	2025-10-14	fengwenxi	加鱼眼；修**中文路径**问题	tool/*.py
`033567b`	2025-09-22	fengwenxi	"fix bug"（工具界面，含 LOGO 资源）	tool/FusionFlow
★ `7a5928c`	2025-06-26	fengwenxi	`LaneNO` → `LaneNo`	factory
★ `5219596`	2025-05-28	lijin1	补 `#include <limits>`	math/util.hpp
★ `3528089`	2025-02-20	fengwenxi	deqing 现场调试：headway/occupancy 改周期累积、加 Gap	6 个文件
★ `19597e3`	2025-02-19	fengwenxi	**经纬度顺序反了**；占用率公式换回累加	vec2d.hpp / occupancy
`63c9b08`	2025-02-18	fengwenxi	注释掉 OccupancyDetector 的空析构	occupancy_detector.cpp

commit	日期	作者	内容	涉及
★ `84b7d07`	2025-02-18	fengwenxi	Volume 只算机动车；分类集合归位	volume_detector
★ `7f71330`	2025-02-18	fengwenxi	**inner_id → id**（跨帧串车）	5 个文件
★ `9b476d1`	2024-12-24	fengwenxi	几何库合并 geo_calc→geometry_calc；错误码定义	17 个文件
★ `47485fb`	2024-12-06	fengwenxi	PCU/Volume 配置来源改 custom_config_json	pcu/volume detector
★ `f1122bb`	2024-10-21	fengwenxi	加检测器 + 完善阻塞队列	13 个文件
★ `a818aa8`	2024-10-17	fengwenxi	**12 个重载合并**；修谓词写反	blocking_queue.hpp
★ `9a3abd9`	2024-10-16	fengwenxi	阻塞队列：active_ 检查位置、加超时版本	blocking_queue.hpp
★ `6b7c6af`	2024-10-15	fengwenxi	阻塞队列：while 丢帧、返回值、退出卡死	blocking_queue.hpp

> 剩下的 `033567b`（工具界面 fix，主要是 UI 细节 + LOGO 资源，无知识点） 和 `63c9b08`（把空析构注释掉，一行）价值不大，diff 在 `fixes/` 里，需要时自己翻。

怎么继续挖

```
# 看某个案例的完整 diff（本文引用的都在这里）
cd D:\ZhiLuOS\61traffic_flow-git历史\fixes
ls                              # 26 个 md，每个含完整 diff
```

```
# 上 193 机查更多（.gitconfig 里已加好 safe.directory）
ssh 172.20.65.193
cd /work/sunboxue/LiuLiang/traffic_flow

git show 997de52                        # 某次提交全文
git log -p -- src/traffic_flow/detectors/accumulate_queue_detector.cpp   # 某文件全部历史
git blame -L 120,170 src/traffic_flow/detectors/accumulate_queue_detector.cpp  # 某几行谁写的
```

```
# 想看某个字段/函数是怎么一步步演化的
git log -S "gap_body_ratio" --oneline --all      # 搜索"引入/删除这个字符串"的提交
git log -G "entry_gap_body_ratio" -p --oneline   # 按正则搜 diff 内容
```

`git log -S`（pickaxe）是挖"这个字段什么时候加的、为什么加"的最好工具，比 blame 更强。